

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Двоичные деревья поиска

Практикум

по выполнению лабораторной работы № 20

по дисциплине "Структуры и алгоритмы обработки данных"

Ростов-на-Дону

20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.

Практикум по выполнению лабораторной работы по дисциплине "Структуры и алгоритмы обработки данных". ДГТУ, г. Ростов на-Дону, 20 г.

В практикуме кратко изложены теоретические вопросы, необходимые для успешного выполнения лабораторной работы, рабочее задание и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлениям 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный за выпуск:

зав. кафедрой Долгов В.В.

Издательский центр ДГТУ, 20

Оглавление

Введение 4

Теоретическая часть 4

1. Базовые концепции 4

2. Представление двоичных деревьев поиска 5

3. Базовые операции 7

4. Обход дерева 9

Инструкция по технике безопасности при проведении работы 11

Цель работы 11

Рабочее задание 11

Общие положения 13

Материально-техническое обеспечение работы 13

Порядок выполнения работы 14

Контрольные вопросы 14

Перечень использованных информационных ресурсов 16

Введение

Двоичные деревья поиска – это иерархические структуры данных, используемые для быстрого доступа к хранящейся в них информации. В лучшем случае эффективность поиска в таких структурах данных может быть оценена как O(log(n)), где n представляет масштаб задачи (количество узлов в дереве), правда в худшем случае эффективность может деградировать до O(n). Для обеспечения возможности быстрого поиска распределение данных по узлам дерева должно подчиняться определенным правилам. Только некоторые операции, в первую очередь операции добавления и удаления узла, могут в принципе привести к нарушению этих правил. Операция поиска нужного узла (точнее информации, содержащейся в этом узле) использует правила для того, чтобы реализующий ее алгоритм был эффективным.

Теоретическая часть

1. Базовые концепции

Двоичные деревья – это упорядоченные деревья, в которых у каждого узла может быть не более двух узлов-потомков, то есть это упорядоченные деревья степени 2. Узлы содержат данные, важные в рамках решаемой задачи. Древовидная структура отображает иерархический, нелинейный характер связей между данными задачи.

Двоичное дерево поиска – это двоичное дерево, каждому узлу которого приписано некоторое значение – ключ. Ключи должны быть сравнимыми, то есть для них должны быть определены операции сравнения на равенство, на неравенство, на меньше, меньше или равно, на больше, больше или равно. Часто требуют, чтобы значения ключей были уникальными, тогда для всякого узла должно выполняться требование – значение его ключа должно быть большим, чем значения ключей его левых потомков и меньшим, чем значения

ключей его правых потомков. Но в общем случае, значения ключей могут повторяться, тогда строгие проверки заменяются на нестрогие – больше на больше или равно, меньше – на меньше или равно. Несмотря на то, что значение ключа может вычисляться по какому-то предопределенному алгоритму, чаще всего оно хранится в самом узле. При этом, помимо ключа, узел может содержать и другую информацию.

Как видно из названия, основной операцией для двоичных деревьев поиска является операция нахождения элемента с заданным ключом. Существует много задач, в которых дерево строится, заполняется нужной информацией и, затем, многократно производится обращение к этой информации, а добавление и удаление новых данных происходит редко. Поиск будет быстрым, если количество узлов в левых поддеревьях не сильно отличается от их количества в правых поддеревьях, тогда операция поиска выполняется очень эффективно (с логарифмической сложностью).

2. Представление двоичных деревьев поиска

Представление деревьев в памяти в общем случае зависит от решаемой задачи. Когда-то, в редких случаях, можно представить дерево как линейную структуру данных – массив или список и при этом иметь возможность создавать эффективные реализации основных операций над деревьями. Примером может служить двоичная пирамида.

Чаще всего для представления деревьев используется цепная реализация, в которой каждый узел, помимо собственно информационной части, содержит также ссылки на дочерние узлы.

Для двоичных деревьев будут храниться две ссылки – на левого и правого потомков. В принципе этого достаточно для реализации основных операций

над деревьями, но для упрощения реализации этих операций, особенно в учебных целях, удобно хранить в каждом узле также ссылку на родительский узел. Таким образом, если в каждом узле должна храниться информация некоторого известного типа T представление для узла двоичного дерева может быть таким

```
record BinaryTreeNode
    data : T
    leftChild, rightChild, parent : BinaryTreeNode^
```

здесь BinaryTreeNode^ обозначает ссылку на узел двоичного дерева.

С деревом можно работать имея ссылку на его корневой узел, то есть в качестве представления для дерева в целом можно выбрать тип BinaryTreeNode^, но в объектно-ориентированном подходе мы для такого важного понятия как двоичное дерево скорее всего создадим собственный тип

```
record BinaryTree
    root : BinaryTreeNode^
```

Очевидно, что представлением для пустого двоичного дерева (EmptyBinaryTree) будет BinaryTree, в котором значение root – это пустая ссылка null.

Для двоичного дерева поиска используется такая же реализация, но с узлом такого дерева должно быть связано некоторое ключевое значение типа K, тогда представление может быть следующим

```
record BSTNode
    key : K
    data : T
```

leftChild, rightChild, parent : BSTNode^

record BST

root : BSTNode^

(обозначение BST - от **B**inary **S**earch **T**ree).

3. Базовые операции

Базовыми операциями для двоичных деревьев и двоичных деревьев поиска являются операции над отдельными элементами деревьев (далее для примера приводятся их описания для двоичных деревьев). Это операции:

1. Проверка на пустоту
2. Добавление нового узла с заданным значением
3. Удаление узла с указанным значением
4. Поиск узла с указанным значением

Трактовка всех операций, кроме первой может быть неоднозначной. Например, добавлять новый узел можно так, чтобы в новом дереве он оказался листом или корнем или внутренним узлом. В общем случае наиболее просто добавить лист. Более того, даже в случае двоичного дерева поиска новый узел всегда можно добавить в качестве листа так, чтобы дерево оставалось двоичным деревом поиска. Поэтому, если не оговаривается обратное, добавление нового узла выполняется как добавление листа. Для двоичного дерева место прикрепления к дереву нового листа можно выбирать по-разному. Например, двигаемся от корня по левой ветке пока не найдем узел, у которого нет левого или правого потомка, в первом случае новый узел становится левым потомком найденного узла, во втором – правым потомком. Для двоичного дерева поиска с уникальными значениями, добавление листа - это однозначно определенная процедура. Двигаясь от корня, переходим к левому потомку,

если ключ добавляемого значения меньше значения ключа корня или к правому потомку, в противном случае. При этом нам нужен такой узел, у которого нет левого потомка, если его ключ больше добавляемого или нет правого потомка, если его ключ больше добавляемого.

Операции удаления и поиска узла становятся неоднозначными, если значения ключей узлов могут повторяться. Тогда обычно имеют в виду первый попавшийся узел с заданным значением ключа.

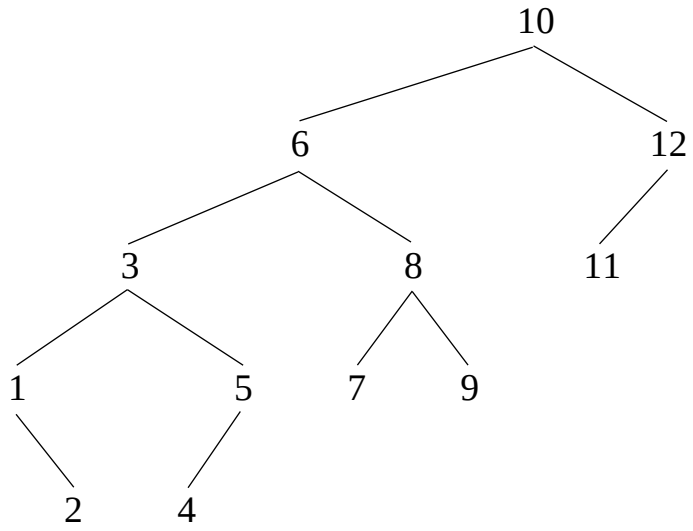


Рисунок 1. Двоичное дерево поиска

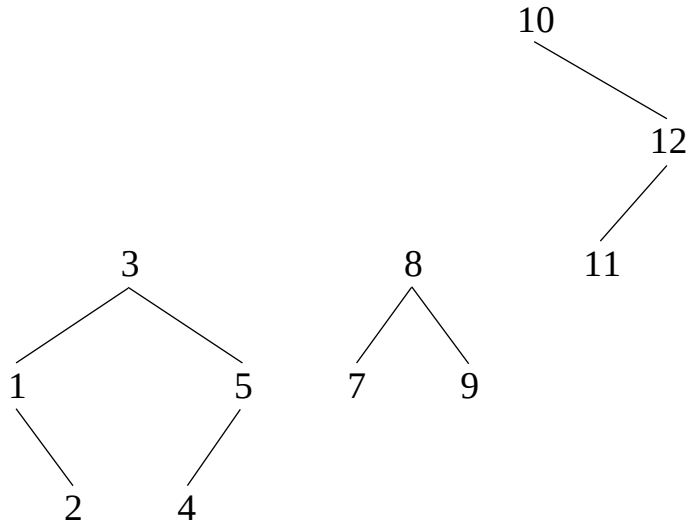


Рисунок 2. После исключения узла 6

Для удаления узла необходимо прежде всего найти его в дереве. Кроме того, нам необходима следующая информация: где расположен узел-предок удаляемого узла, каким (левым или правым) потомком является удаляемый узел для узла-предка, где расположены узлы-потомки удаляемого узла. Например, пусть для двоичного дерева поиска на рисунке 1, удаляемый узел – это узел 6, тогда родительский – узел 10, причем удаляемый узел 6 – левый потомок 10, левый потомок узла 6 – это узел 3 (точнее дерево,

корнем которого является узел 3), правый потомок – узел 8 (поддерево с корнем 8). Если исключить узел 6, получим три дерева, рисунок 2. Нужно объединить их в одно. Для этого можно, например, поддерево с корнем 8 сделать самым правым (в нем ключи всех узлов больше) поддеревом дерева с корнем 3. Теперь остается сделать все новое дерево с корнем 3 левым (в нем все ключи меньше) поддеревом исходного дерева с корнем 10, рисунок 3. Очевидно, что можно поступить и иначе. Вначале сделать поддерево с корнем 3 самым левым поддеревом дерева с корнем 8, а, затем, получившееся дерево с корнем 8 сделать левым поддеревом дерева с корнем 10.

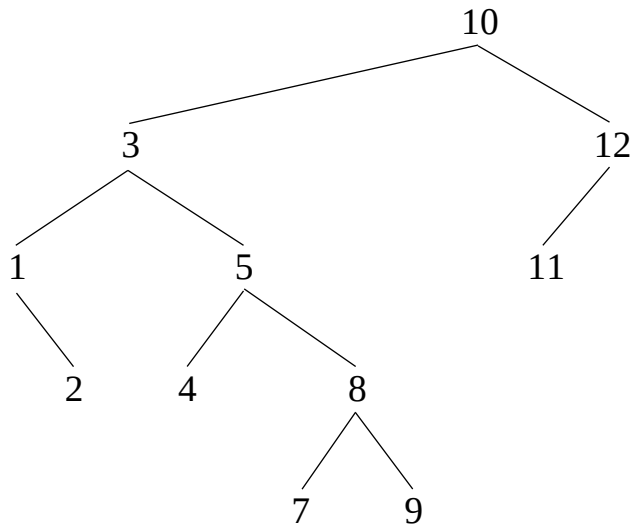


Рисунок 3. Новое дерево

4. Обход дерева

Под обходом дерева подразумевают получение значений всех узлов дерева в виде некоторой линейной последовательности. В зависимости от решаемой задачи, полученные в результате обхода значения могут выводиться на экран, помещаться в файл, список или массив. Порядок следования полученных значений зависит от алгоритма обхода. Для деревьев традиционно рассматривают три варианта обхода в глубину и один вариант обхода в ширину.

Обход в глубину. Рассмотрим описания рекурсивных вариантов алгоритмов.

1. В прямом порядке.

Вначале получаем значение корня дерева (выводим на экран, помещаем в файл, добавляем в список или массив), затем в прямом порядке обходим левое поддерево и, в последнюю очередь, обходим в прямом порядке правое поддерево. Для двоичного дерева поиска на рисунке 4, получим следующую последовательность: 6, 3, 2, 1, 5, 4, 7.

2. Симметричный.

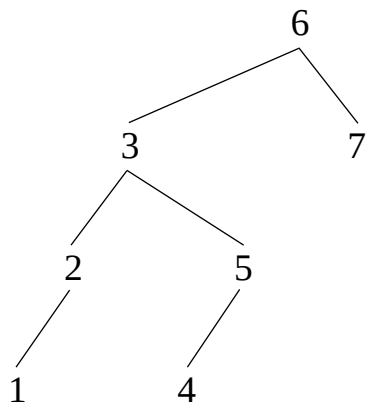


Рисунок 4.

Вначале симметричным образом обходим левое поддерево, затем получаем значение корня дерева (выводим на экран, помещаем в файл, добавляем в список или массив), и, в последнюю очередь, обходим симметричным образом правое поддерево. Для двоичного дерева поиска на рисунке 4, получим следующую последовательность: 1, 2, 3, 4, 5, 6, 7. В случае двоичного дерева поиска такой обход дает упорядоченную последовательность.

3. В обратном порядке.

Вначале в обратном порядке обходим левое поддерево, затем обходим в обратном порядке правое поддерево и, в последнюю очередь, получаем значение корня дерева (выводим на экран, помещаем в файл, добавляем в список или массив). Для двоичного дерева поиска на рисунке 4, получим следующую последовательность: 1, 2, 4, 5, 3, 7, 6.

Для обхода в глубину можно построить нерекурсивные алгоритмы. В этих алгоритмах в процессе прохода по дереву (в нужном порядке) ссылки на узлы помещаются в дополнительную структуру данных – стек. Например, для обхода в прямом порядке вначале помещаем в стек ссылку на корень, затем

пока стек не пуст повторяем следующие действия: извлекаем из стека верхний элемент, выводим значение его ключа, если есть правый (именно правый, так как из стека извлекается не первый, а последний добавленный элемент) потомок – помещаем в стек ссылку на его корень, если есть левый потомок – помещаем в стек ссылку на его корень.

В алгоритме обхода в ширину используется нерекурсивный алгоритм, в котором вместо стека для хранения ссылок на узлы используется очередь.

Инструкция по технике безопасности при проведении работы

При выполнении работы нужно выполнять требования по технике безопасности при работе с электронными приборами.

Цель работы

Изучение представления и базовых операций для двоичных деревьев и двоичных деревьев поиска.

Рабочее задание

ЗАДАНИЕ 1. Создайте реализацию структуры данных "Двоичное дерево". Можно ограничиться целыми значениями элементов (ключей), но, если использованный Вами язык программирования позволяет, лучше создать обобщенную версию, в которой ключи могут быть любого подходящего типа. Должны поддерживаться следующие основные операции: проверка на пустоту, добавление нового узла в качестве листа дерева, удаление указанного узла, поиск узла с заданным значением (возвращает ссылку на соответствующий узел), проверка наличия в дереве узла с заданным значением (возвращает

булево значение), определение высоты заданного узла, определение высоты дерева, подсчет количества узлов в дереве, вывод дерева на экран. Кроме того, реализуйте операции обхода дерева в прямом, симметричном, обратном порядке и в ширину. Для обхода в глубину реализуйте как рекурсивную, так и нерекурсивную версии.

ЗАДАНИЕ 2. Создайте реализацию структуры данных "Двоичное дерево поиска" на основе структуры данных "Двоичное дерево" (должны поддерживаться все основные операции, определенные для двоичных деревьев). Все элементы должны быть уникальными.

ЗАДАНИЕ 3. Создайте реализацию структуры данных "Двоичное дерево поиска", в котором значения элементов могут повторяться.

ЗАДАНИЕ 4. Создайте реализацию структуры данных "Множество" (Set) для работы с целыми числами на основе структуры данных "Двоичное дерево поиска с уникальными элементами". Множество поддерживает следующие операции: проверка на пустоту, добавление нового элемента, удаление элемента с указанным значением, проверка на принадлежность значения множеству, пересечение, объединение, дополнение для двух множеств.

ЗАДАНИЕ 5. Создайте реализацию структуры данных "Мультимножество" (Multiset) для работы с целыми числами на основе структуры данных "Двоичное дерево поиска с повторяющимися элементами". Операции такие же как в задании 4.

ЗАДАНИЕ 6. Создайте реализацию структуры данных "Ассоциативный массив" (Map), которую называют также "Отображение", "Словарь" для работы с парами – ключ-значение на основе структуры данных "Двоичное дерево поиска с уникальными элементами". Для этого помимо ключа нужно добавить в каждый узел еще один параметр.

ЗАДАНИЕ 7. Добавить в реализации двоичного дерева поиска операции разбиения на два дерева по узлу с указанным значением и объединения двух деревьев в одно. Элементы уникальные.

ЗАДАНИЕ 8. Добавить к реализации задания 7 операции добавления нового узла в качестве корня и в качестве внутреннего узла.

ЗАДАНИЕ 9. Реализовать задания 7, 8 для деревьев с повторяющимися элементами.

ЗАДАНИЕ 10. Для деревьев с уникальными элементами реализуйте операцию, получающую двоичное дерево и возвращающую булево значение, показывающее является ли данное дерево двоичным деревом поиска.

Общие положения

Двоичные деревья используются во многих алгоритмах для представления иерархически связанных данных. Двоичные деревья поиска при правильной организации позволяют находить хранящуюся в их узлах информацию за время пропорциональное логарифму по основанию два от общего числа элементов. В настоящей работе рассматриваются основные операции, которые должны быть заданы в реализациях таких структур данных.

Материально-техническое обеспечение работы

Компьютеры с установленным программным обеспечением:

1. Java SDK 11+.

2. IDE для работы с корпоративными Java-проектами - желательно IntelliJ Idea Enterprise.

3. Сервер приложений с поддержкой Java EE/Jakarta EE - желательно Apache TomEE+ 8+.

4. Сервер реляционных баз данных (СУБД), например, PostgreSQL (опционально).

5. ConnectorJ - драйвер для работы с сервером реляционных баз данных (например, PostgreSQL) из Java-программ (опционально).

Порядок выполнения работы

1. Ознакомиться с теоретическим материалом.

2. Выполнять задания в порядке их следования так как при выполнении текущего задания потребуются результаты, полученные при выполнении предыдущих заданий.

Контрольные вопросы

1. Что такое двоичное дерево?
2. Какие операции являются базовыми для двоичных деревьев?
3. Каковы основные варианты добавления нового узла в двоичное дерево?
4. Какой вариант добавления нового узла наиболее прост в реализации?
5. Как реализуется алгоритм добавления листа в двоичное дерево?
6. Какие есть варианты обхода двоичного дерева в глубину?
7. В чем заключается обход двоичного дерева в ширину?
8. Как выполнить обход двоичного дерева в прямом порядке?
9. Как выполнить симметричный обход двоичного дерева?

10. Как выполнить обход двоичного дерева в обратном порядке?

11. Как реализуется нерекурсивная версия алгоритма обхода двоичного дерева в прямом порядке?

12. Как реализуется нерекурсивная версия алгоритма симметричного обхода двоичного дерева?

13. Как реализуется нерекурсивная версия алгоритма обхода двоичного дерева в обратном порядке?

14. Что такое двоичное дерево поиска?

15. Для чего используются двоичные деревья поиска?

16. Какие операции являются базовыми для двоичных деревьев поиска?

17. Как реализуется алгоритм добавления листа в двоичное дерево поиска?

18. Какова оценка эффективности операции поиска в двоичном дереве поиска в лучшем случае?

19. Какова оценка эффективности операции поиска в двоичном дереве поиска в худшем случае?

20. Какой алгоритм можно использовать для проверки, является ли заданное двоичное дерево двоичным деревом поиска?

Перечень использованных информационных ресурсов

1. П.Луридас "Алгоритмы для начинающих: теория и практика для разработчика". – М.: Эксмо. – 2018, 608 с.
2. А.Левитин "Алгоритмы: введение в разработку и анализ". – М.: ООО "И.Д. Вильямс". – 2018, 576 с.
3. Н.Вирт "Алгоритмы и структуры данных". – М.: ДМК Пресс. – 2016, 272с.
4. "Алгоритмы в задачах и примерах: самые эффективные способы и решения от сообщества Stack Overflow". – М.: Издательство АСТ. – 2024, 256 с.
5. А.В.Ахо, Д.Д.Ульман, Д.Э.Хопкрофт "Структуры данных и алгоритмы". – М.: ООО "И.Д. Вильямс". – 2016, 400 с.
6. Т.Х.Кормен, Ч.И.Лейзерсон, Р.Л.Ривест, К.Штайн "Алгоритмы: построение и анализ", 3-е изд. – СПб.: ООО "Диалектика". – 2019, 1328 с.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.